SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 2 – Architecture Design Assignment

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO2	**Assessment:**	Assessment Tool 2 – Architecture Design Assignment
Weightage:	20%	**Total Marks:**	25
CO2 Statement:	*Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills*		
Student Name:	U. Kavya	**Reg. No.:**	192472114
Date:	25-10-2026	**Duration:**	60 minutes

Code of Conduct

*I __**U. Kavya (192472114)**__ (Name / Reg No) certifies that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: __U. Kavya_

Annexure A – Architecture Design Assignment

Instructions to Students:

Each student will be assigned an **individual software project problem statement**. Based on the assigned problem, analyze the system requirements and design an appropriate software architecture by applying software engineering principles. Your submission should demonstrate your ability to select, justify, and evaluate a suitable architectural style.

Question

Based on your **assigned problem statement**, perform an **Architecture Design Analysis** and prepare an architecture design report by answering the following questions:

1. Analyze System Requirements

- Study the given problem statement and identify the system objectives.
- Analyze the functional and non-functional requirements that influence the software architecture.
- Identify key quality attributes such as scalability, maintainability, reliability, availability, performance, and security.

2. Select a Suitable Software Architecture

- Select an appropriate architectural style for the proposed system (e.g., Layered Architecture, Client-Server, MVC, Microservices, Event-Driven, Service-Oriented Architecture, Serverless, or Hybrid Architecture).
- Justify why the selected architecture is the most suitable for the given problem.

3. Draw the Software Architecture Diagram

- Design a clear architecture diagram illustrating the overall structure of the proposed system.
- Include all major layers/components, databases, external systems, APIs, and communication mechanisms.
- Use appropriate software architecture notations and labels.

4. Explain Components and Their Interactions

- Describe the purpose and responsibilities of each architectural component.
- Explain how the components communicate and exchange data.
- Illustrate the overall workflow of the system.

5. Discuss Quality Attributes
Evaluate how the proposed architecture addresses the following aspects:

- Scalability
- Security
- Performance
- Reliability
- Maintainability
- Availability

6. Justify Architectural Decisions

- Explain the rationale behind your architectural choices.
- Discuss the trade-offs considered while selecting the architecture.
- Explain how the chosen architecture supports future enhancements, system integration, and long-term maintainability.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
System Requirement Analysis (30%)	Thoroughly analyzes the problem statement, accurately identifies functional and non-functional requirements, and clearly explains quality attributes influencing the architecture.	Analyzes most requirements with minor omissions or limited explanation.	Identifies only basic requirements with partial analysis.	Requirement analysis is incomplete, inaccurate, or lacks clarity.
Selection of Software Architecture (25%)	Selects the most appropriate architectural style with strong technical justification aligned to system requirements.	Selects a suitable architecture with reasonable justification.	Architecture is partially suitable with limited justification.	Inappropriate architecture selected or justification is missing.
Architecture Diagram and	Produces a clear, complete, and well-structured	Diagram is mostly complete with	Diagram includes only essential	Diagram is incomplete, unclear, or

Component Design (20%)	architecture diagram with all major components, interfaces, and interactions accurately represented.	minor omissions or notation issues.	components and lacks sufficient detail.	technically incorrect.
Component Interaction and Quality Attribute Analysis (15%)	Clearly explains component responsibilities, interactions, and thoroughly discusses scalability, security, performance, reliability, maintainability, and availability.	Explains most components and quality attributes with minor gaps.	Provides limited explanation of interactions and addresses only some quality attributes.	Component interactions and quality attributes are poorly explained or missing.
Architectural Justification and Technical Presentation (10%)	Provides strong justification for architectural decisions, discusses trade-offs and future enhancements, and presents a well-organized, professional report.	Provides adequate justification with minor gaps; report is well organized.	Limited justification with minimal discussion of trade-offs or future scope; report needs improvement.	Justification is weak or absent; report lacks organization and technical clarity.

Criteria	**Mark**
System Requirement Analysis **(30%)**	/5
Selection of Software Architecture **(25%)**	/5
Architecture Diagram and Component Design **(20%)**	/5
Component Interaction and Quality Attribute Analysis **(15%)**	/5
Architectural Justification and Technical Presentation **(10%)**	/5
TOTAL	/25

1. Analyze System Requirements

1.1 System Objectives

The platform provides end-to-end container security from image creation through production execution. It prevents vulnerable or misconfigured images from reaching production and continuously detects abnormal behaviour in running workloads.

- Scan container images for vulnerabilities, secrets, malware indicators, and insecure dependencies.
- Validate Dockerfiles, Kubernetes manifests, Helm charts, and runtime configurations.
- Monitor runtime processes, system calls, file activity, network behaviour, and Kubernetes audit events.
- Detect suspicious activities using policies, behavioural baselines, and anomaly detection.
- Integrate security gates with CI/CD pipelines.
- Support Kubernetes orchestration and centralized logging.
- Provide incident-response, alerting, forensic evidence, and audit mechanisms.

1.2 Functional Requirements

FR-1: Scan container images and dependencies for known vulnerabilities.

FR-2: Detect embedded secrets, private keys, and malware indicators.

FR-3: Validate container and Kubernetes configurations against security policies.

FR-4: Provide CI/CD pass/fail decisions based on scan and policy results.

FR-5: Collect runtime telemetry from containers and Kubernetes nodes.

FR-6: Create behavioural baselines for normal workload activity.

FR-7: Detect suspicious events such as privilege escalation, unexpected shells, sensitive-file access, and unusual outbound traffic.

FR-8: Generate alerts and support containment or remediation actions.

FR-9: Aggregate logs, events, and audit records centrally.

FR-10: Retain immutable audit records and generate compliance reports.

FR-11: Provide dashboards and APIs for security and DevOps teams.

1.3 Non-Functional Requirements and Quality Attributes

- Scalability: Support many clusters, nodes, namespaces, and containers.
- Performance: Provide fast image scans and near-real-time runtime detection.
- Reliability: Avoid loss of security telemetry during failures.
- Availability: Keep monitoring and alerting available during component failures.
- Security: Protect platform APIs, telemetry, secrets, reports, and audit data.
- Maintainability: Enable independent updates to scanners, policies, and detection rules.
- Extensibility: Support new orchestrators, registries, cloud providers, and detection sources.

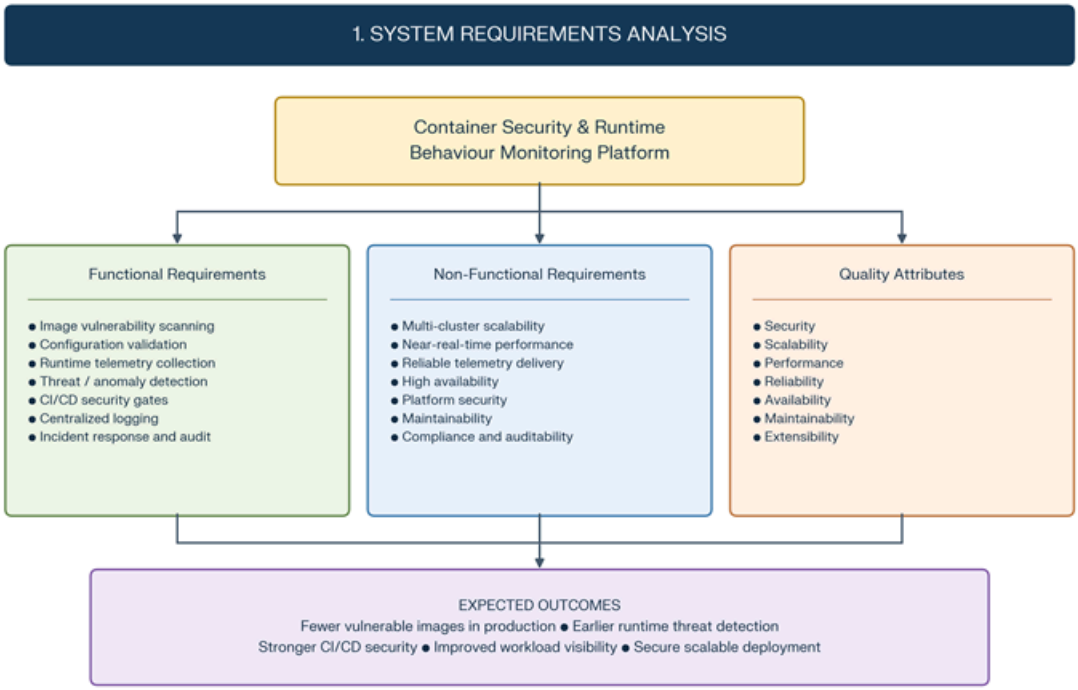


Figure 1: Requirements analysis and quality-attribute map.

2. Select a Suitable Software Architecture

A Hybrid Architecture combining Microservices, Event-Driven Architecture, and Layered Architecture is selected. Synchronous APIs support CI/CD gates, dashboards, and policy decisions, while asynchronous event streams handle high-volume runtime telemetry, alerting, logging, and audit processing.

- Microservices allow image scanning, configuration validation, runtime analysis, incident response, and reporting to scale independently.
- Event-driven architecture handles continuous runtime events without coupling sensors to analysis services.
- A layered structure separates presentation, API, security services, and data storage.
- Kubernetes-native deployment supports autoscaling, self-healing, and centralized orchestration.

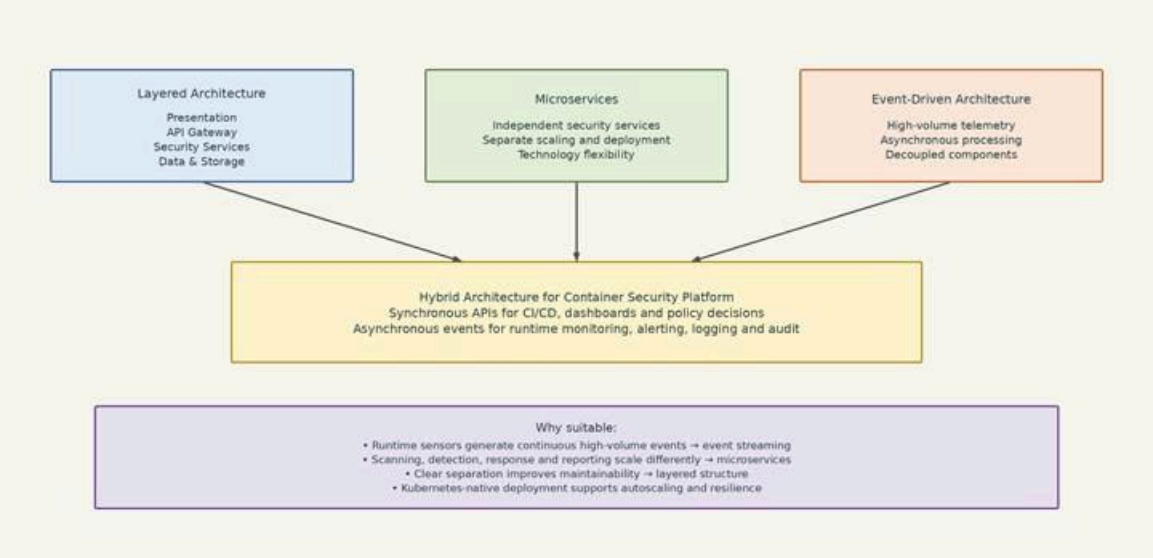


Figure 2: Architectural-style selection and justification.

3. Software Architecture Diagram

The architecture includes user-facing dashboards, an API gateway, security microservices, an event backbone, runtime sensors, data stores, Kubernetes clusters, container registries, and CI/CD systems.

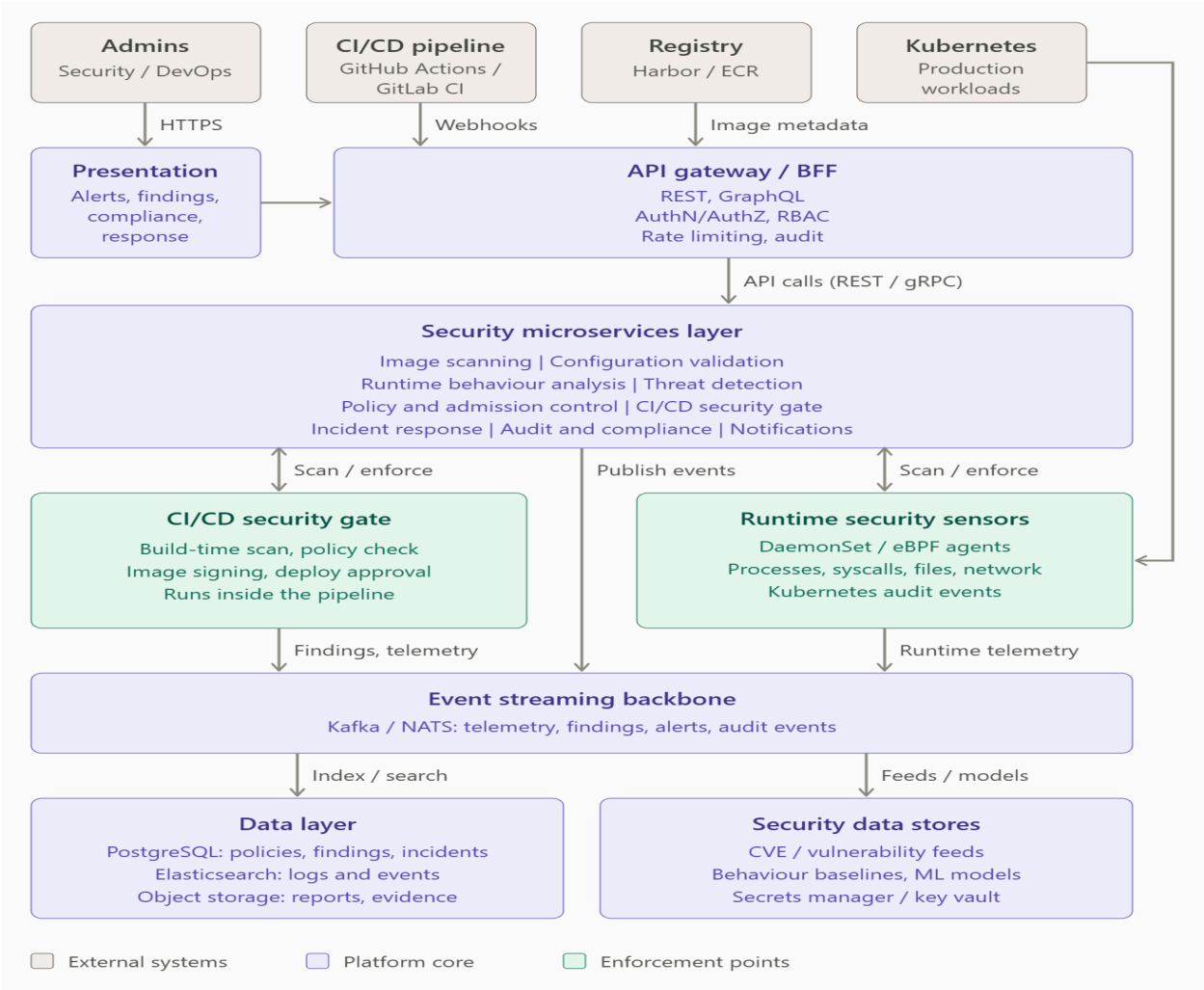


Figure 3: Overall hybrid microservices and event-driven architecture.

4. Components and Their Interactions

4.1 Major Components

- Presentation Layer: Dashboard for vulnerability findings, runtime alerts, compliance status, and incident response.
- API Gateway: Secure entry point providing authentication, authorization, rate limiting, routing, and audit logging.
- Image Scanning Service: Identifies CVEs, outdated dependencies, secrets, and malware indicators.

- Configuration Validation Service: Checks Dockerfiles, Kubernetes manifests, and Helm charts against security policies.
- Runtime Behaviour Analysis Service: Compares live workload activity with learned baselines.
- Threat Detection Engine: Applies correlation rules, signatures, and anomaly detection.
- Policy and Admission Control Service: Blocks unsigned, unscanned, or policy-violating deployments.
- CI/CD Security Gate Service: Returns pass/fail decisions and evidence to CI/CD systems.
- Incident Response Service: Manages alert triage, containment, remediation, and post-incident reports.
- Audit and Compliance Service: Retains immutable records and generates compliance evidence.
- Runtime Security Sensors: Collect process, syscall, file, network, and Kubernetes audit events.
- Event Streaming Backbone: Kafka or NATS transports telemetry, findings, alerts, and audit events.
- Data Layer: PostgreSQL for structured data, Elasticsearch for logs/events, and object storage for reports and evidence.

4.2 Interaction Workflow

12. A developer commits code and container configuration.
13. CI/CD builds an image and requests vulnerability scanning.
14. The scanner returns CVEs, secrets, malware indicators, and a risk score.
15. The configuration validator checks deployment manifests and runtime settings.
16. The security gate evaluates scan results, signatures, and policies.
17. Non-compliant images are blocked; compliant images are signed and deployed.
18. Kubernetes admission control performs a final policy check.
19. Runtime sensors collect workload telemetry and publish it to the event backbone.
20. Detection services analyze events and generate prioritized alerts.
21. Security teams investigate and respond through the dashboard.
22. All findings, decisions, and response actions are logged for audit.

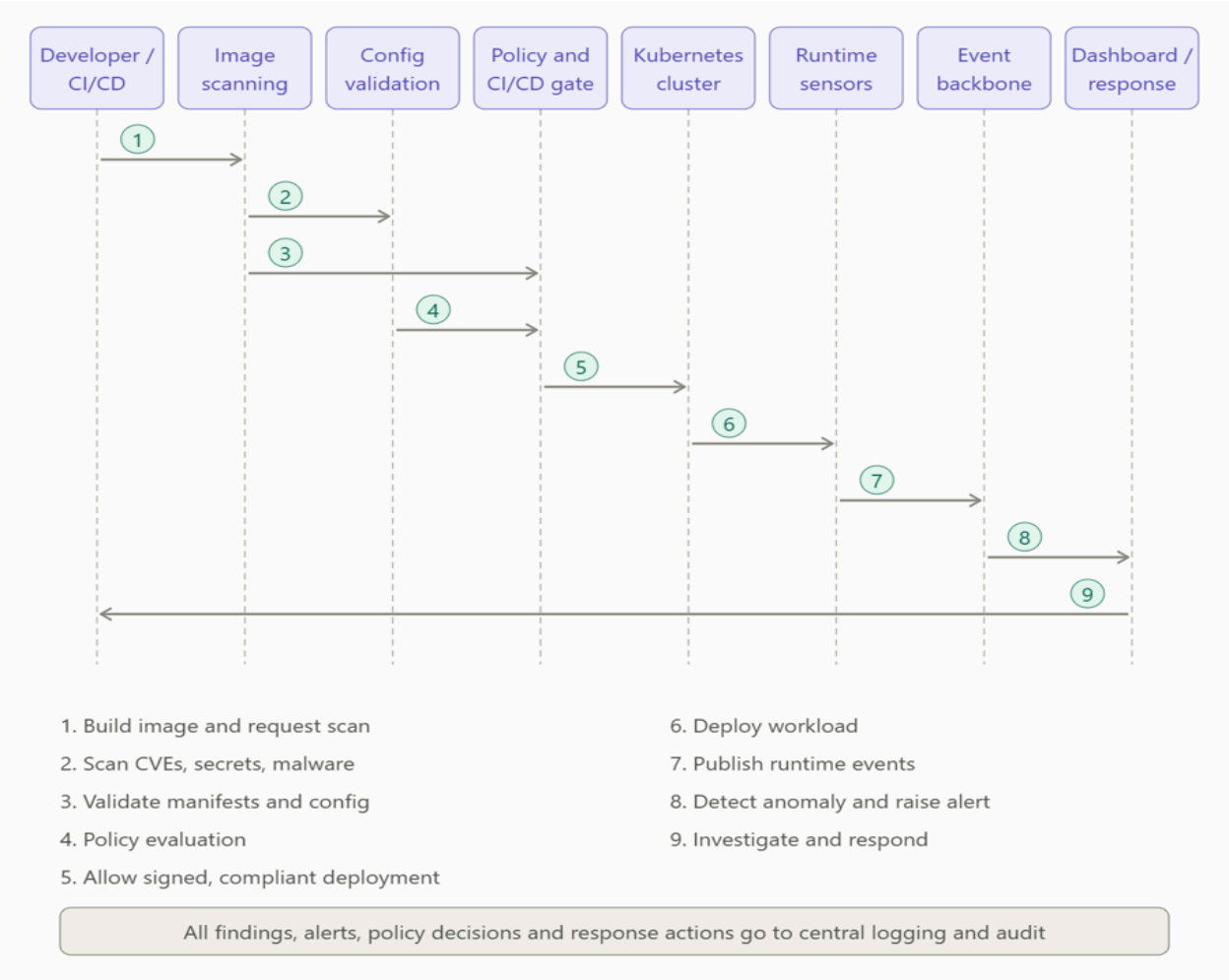


Figure 4: Component interaction and end-to-end security workflow.

5. Discuss Quality Attributes

- Scalability: Stateless services scale horizontally; Kafka partitions parallelize telemetry; DaemonSet sensors scale with nodes.
- Security: RBAC, TLS/mTLS, encryption, secrets management, least privilege, and immutable audit logs protect the platform and its data.
- Performance: Asynchronous processing, caching, indexing, and alert prioritization reduce latency.
- Reliability: Message queues, retries, local sensor buffering, and replicated storage prevent data loss.

- Maintainability: Independent services, policy-as-code, and pluggable scanners simplify updates.
- Availability: Kubernetes replicas, health probes, autoscaling, and multi-zone deployment maintain service continuity.

Quality attribute	Architectural mechanism	How the platform achieves it
Scalability	Horizontal scaling, Kafka partitions, DaemonSet sensors	Scale scanning, detection and API services independently; sensors scale with nodes
Security	RBAC, mTLS, encryption, secrets management, audit logs	Protect APIs, telemetry, credentials and findings; enforce least privilege
Performance	Async processing, caching, indexing, prioritized alerts	Avoid blocking CI/CD and workloads; process critical events first
Reliability	Queues, retries, local buffering, replication	Prevent telemetry loss during service or network failures
Maintainability	Microservices, policy-as-code, plugin scanners	Update rules, policies and integrations independently
Availability	Kubernetes replicas, probes, autoscaling, multi-zone deployment	Keep monitoring services available during failures and maintenance

Figure 5: Quality attributes and corresponding architectural controls.

6. Justify Architectural Decisions

- Microservices were selected because security capabilities have different scaling and release cycles.
- Event-driven processing was selected because runtime monitoring produces continuous high-volume telemetry.
- Kubernetes-native sensors provide close integration with nodes, pods, container runtimes, and audit events.

- Policy-as-code makes security controls version-controlled, testable, reusable, and consistently enforced.
- Centralized logging supports investigation, correlation, compliance, and long-term visibility.

The main trade-offs are higher operational complexity, sensor resource consumption, potential CI/CD delays from strict gates, and privacy considerations for runtime telemetry. These are mitigated through managed event infrastructure, resource limits, risk-based policies, approved exceptions, data minimization, RBAC, and observability.

The architecture supports future enhancements such as machine-learning anomaly detection, SIEM/SOAR integration, SBOM generation, artifact signing, multi-cloud adapters, and additional compliance mappings without redesigning the platform.

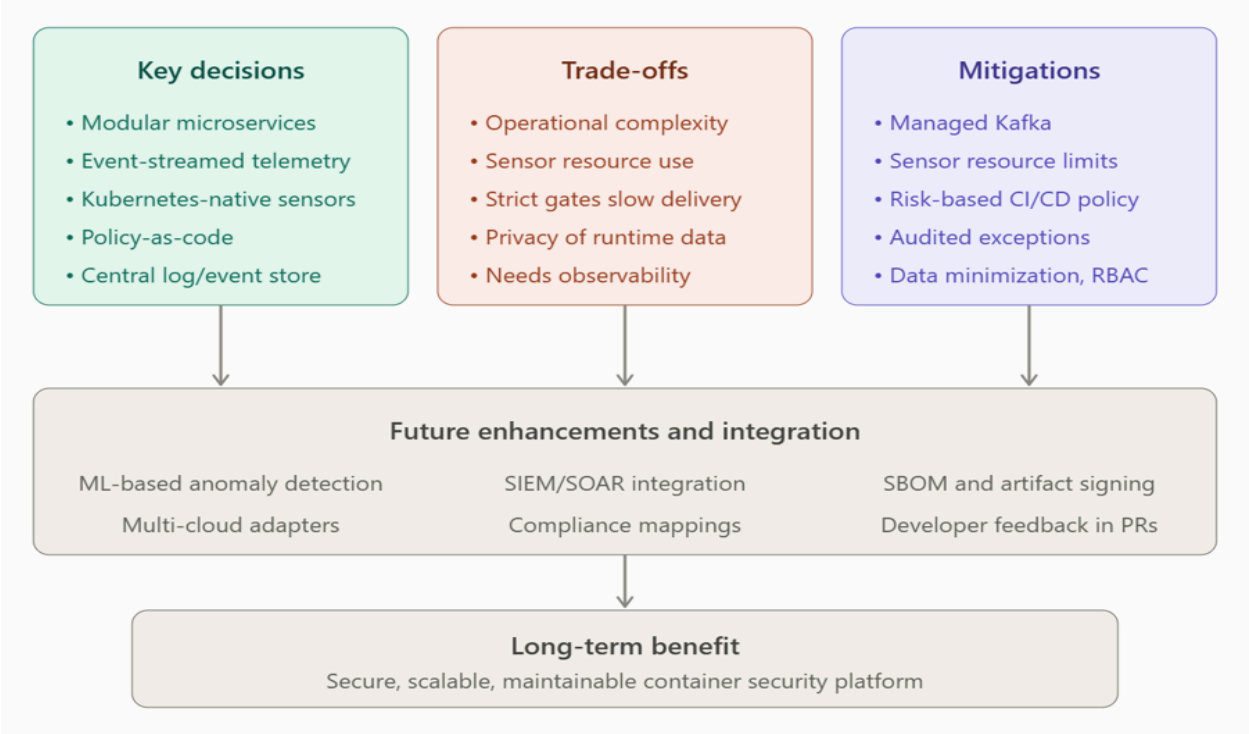


Figure 6: Architectural decisions, trade-offs, mitigations, and future enhancements.